

A Motion Planning Framework for Multi-Robot Navigation in ROS 2

◦Luis Contreras* (QibiTech Inc.), Marco Negrete (National Autonomous University of Mexico),
Tomoaki Yoshikai (QibiTech Inc.), Yuichiro Hirose (QibiTech Inc.),
and Hiroyuki Okada (東京情報デザイン専門職大学)

Abstract—Domestic service robots are increasingly being deployed in public environments, where efficient navigation is essential. As multiple robots operate in shared spaces, communication and coordination become crucial for task performance. This paper presents a complete motion planning framework for ROS 2-based multi-robot systems. It integrates spatial representation, path and motion planning algorithms, and behavior-based active navigation.

I. INTRODUCTION

A service robot performs useful tasks for humans or equipment outside of industrial automation and requires a certain level of autonomy [1]. While full autonomy is not a necessity, a higher degree of autonomy significantly increases the robot’s usefulness, particularly in scenarios like space or domestic applications where human assistance is limited or undesirable. Domestic service robots must integrate various capabilities such as motion planning in dynamic environments, object recognition, manipulation, and natural human-robot interaction. Among these, motion planning remains a critical and complex task, involving the robot’s ability to localize, map, and navigate safely within its environment.

Motion planning systems are composed of interdependent components—such as localization, path planning, and control—where performance is often impacted by feedback loops, making holistic system evaluation essential. Robotic competitions like RoboCup [2] and World Robot Summit [3] provide valuable platforms to benchmark complete robotic systems under controlled and replicable conditions, uncovering challenges not evident in isolated experiments [4]. Despite significant research and available open-source navigation libraries, motion planning in cluttered, real-world environments continues to be a challenging problem. This is evident in recent RoboCup@Home tasks, which test robots on their ability to navigate around difficult-to-detect obstacles like small objects or transparent items, highlighting the ongoing need for innovation in autonomous navigation.

This paper presents the design and implementation of a modular navigation system built on ROS 2. The system integrates map augmentation, sensor fusion, path planning, and motion control through a multi-node architecture. We describe each component and its configuration and discuss its flexibility for various robotic platforms.

In contrast to existing frameworks like Nav2, which support obstacle source separation through static configuration, our system allows for dynamic integration of new sensor inputs without requiring configuration file rewrites or node restarts. This makes it especially suitable for dynamic environments or heterogeneous robot teams where sensor configurations may vary at runtime. Furthermore, the architecture is designed with extensibility in mind, enabling seamless integration of semantic representations, learning-based decision-making, and 3D navigation capabilities—critical features for public-facing applications such as crowded environments or plant inspection scenarios.

II. RELATED WORK

This paper presents a ROS 2 implementation of [5], and is based on the ROS 1 implementation as in [6].

Popular frameworks such as ROS 2 Navigation Stack (Nav2) provide modular and configurable solutions for localization, planning, and control. However, they often assume static configurations and are primarily designed for single-robot systems operating in relatively stable environments. Our approach diverges by targeting flexibility and runtime adaptability. It supports multi-robot scenarios, handles real-time sensor reconfiguration, and is extensible to incorporate semantic mapping, 3D spatial awareness, and reinforcement learning techniques. These capabilities are essential for deploying service robots in complex, dynamic settings.

III. SYSTEM OVERVIEW

The developed autonomous navigation system consists of seven modules: route planner, map manager, low-level controller, collision risk detector, and movement planner. These modules interact with each other to perform autonomous navigation to a target point with obstacle avoidance. In addition to the developed modules, the navigation system integrates two third-party modules: the localizer (`amcl` package) and the map server (`map_server` package). The system is designed to utilize the sensors and actuators available on the target robots. Sensor inputs can be dynamically added or reconfigured at runtime, allowing the system to adapt to new sensor types or failure scenarios without redeployment. The supported sensors include LIDAR and an RGB-D cameras (capable of providing 3D point clouds). Motion planning assumes a differential-drive mobile base.

The user can set target points by publishing to the `/move_base_simple/goal` topic. The user can stop the robot’s movement at any time by publishing to the

*l.contreras@qibitech.com

◦ Presenter

ACKNOWLEDGMENT: This work was supported by the project, JPNP16007, commissioned by the New Energy and Industrial Technology Development Organization (NEDO) and by UNAM-DGAPA under grant PAPIIT IT102424.

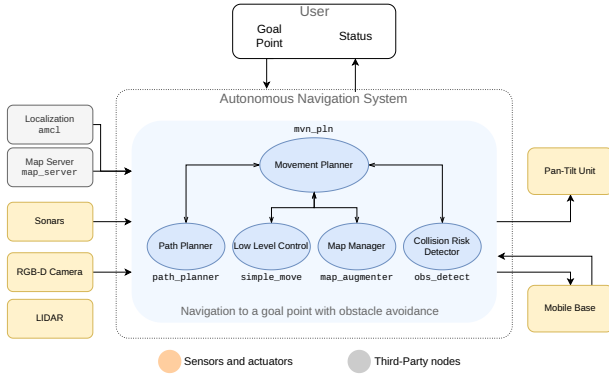


Fig. 1: General architecture of the navigation system

/stop topic, and monitor the system status through the /navigation/status topic.

Figure 1 shows a block diagram of the developed system. For autonomous navigation, at least one LIDAR sensor is required, since localization relies on it. The RGB-D camera is optional; their use improves obstacle avoidance. The navigation system generates motion commands for the mobile base and, optionally, for the head.

IV. IMPLEMENTATION DETAILS

A. Nodes and Functionality Summary

This section groups the nodes by their functional roles within the navigation system.

Standard Motion Planning packages:

Mapping:

- **static_map_server** (nav2_map_server): Loads the static environment map from a YAML file and publishes it as /nav_map.
- **prohibition_map_server** (nav2_map_server): Loads a constraint map (e.g., areas robots must avoid) and publishes it as /fixed_prohibition_layer_map.
- **lifecycle_manager_map** (nav2_lifecycle_manager): Manages the lifecycle of map-related nodes, ensuring proper initialization and shutdown.

Map Enhancement:

- **map_enhancer** (augment_gridmap_online): Dynamically augments the prohibition layer map by adding user-defined obstacles, e.g., via RViz2 interactive markers.
- **map_augmenter** (map_augmenter): Integrates data from multiple sensors (laser scan, point clouds) to enhance the environment representation with dynamic obstacles. Parameters allow tuning filtering thresholds and spatial bounds.

Path Planning:

- **path_planner** (path_planner): Computes a global path from the robot's current position to the target using the enhanced map. Optional support for diagonal movement.

- **mvn_pln** (mvn_pln): Acts as the main navigation planner, receiving user goals and ensuring path execution. Remaps /move_base_simple/goal.

Motion and Control:

- **simple_move** (simple_move): Executes movement commands based on the planned trajectory, with tunable parameters for linear and angular speed, control gain, and optional use of potential fields.
- **potential_fields** (potential_fields): Implements local obstacle avoidance using artificial potential fields derived from laser and point cloud data. Parameters control interaction ranges, repulsion strength, and input source selection.
- **publish_enable** (potential_fields): Publishes a toggle or enable flag, likely used to activate or deactivate control modules (e.g., potential fields).

B. Nodes and Functionality

Standard Packages:

static_map_server: We use a standard map server to publish a static map.

- **Package:** nav2_map_server
- **Parameters:** yaml_filename

path_planner: This node plans paths using the A* algorithm and then applies smoothing via gradient descent.

- **Package:** path_planner
- **Parameters:** diagonal_paths

simple_move: This node implements a nonlinear controller for path following. In the control law, the desired positions are given by the points on the planned path, and the control output consists of movement commands for the mobile base. This node is also responsible for halting motion if a collision risk is detected.

- **Package:** simple_move
- **Parameters:** max_linear_speed, max_angular_speed, control_alpha, control_beta, move_head, use_pot_fields

mvn_pln: This node coordinates the execution of autonomous navigation toward a target point with obstacle avoidance through a state machine.

- **Package:** mvn_pln
- **Parameters:** patience

Additional Packages:

prohibition_map_server: We use a standard map server to publish a prohibition map that can be edited online by the map_enhancer node.

- **Package:** nav2_map_server
- **Parameters:** yaml_filename

map_enhancer: Takes the prohibition map and adds obstacles online through a PointStamped value in the /clicked_point topic.

- **Package:** augment_gridmap_online
- **Parameters:** obstacle_radius, input_map

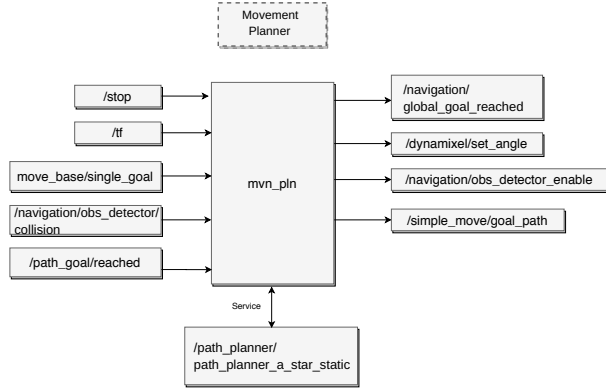


Fig. 2: Block diagram of the `mvn_pln` module, responsible for coordinating autonomous navigation through a state machine that integrates path planning, obstacle detection, and low-level control.

`map_augmenter`: This node uses the selected sensors (LIDAR or point cloud) to generate an augmented map that includes the detected obstacles. Additionally, it creates cost maps both with and without considering obstacles. These maps are used by the route planner.

- **Package:** `map_augmenter`
- **Parameters:** `laser_scan_topic`, `point_cloud_topic`, `point_cloud_topic2`, `static_map_server`, `prohibition_map_server`, `use_lidar`, `use_point_cloud`, `use_point_cloud2`, `cloud_downsampling`, `inflation_radius`, `cost_radius`, `laser_max_x/y/z`, `laser_min_x/y/z`, `cloud_max_x/y/z`, `cloud_min_x/y/z`

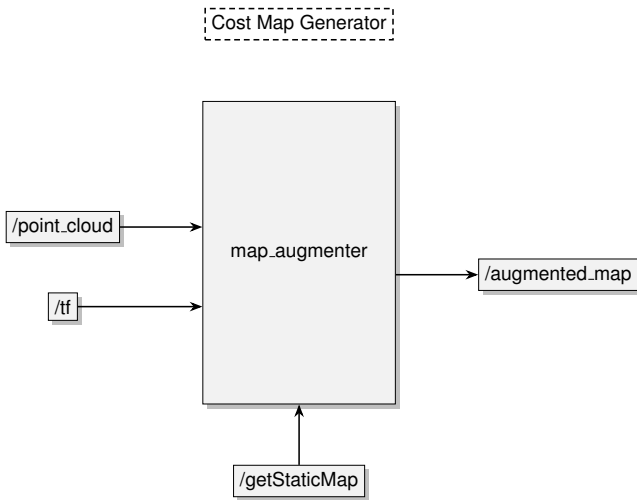


Fig. 3: Block diagram of the `map_augmenter` module. The node processes LIDAR or point cloud data to detect dynamic obstacles and generates an augmented cost map.

`potential_fields`: This node uses the selected sensors (LIDAR or point cloud) to detect potential collision

risks. A risk is determined if the number of sensor readings within a predefined bounding box exceeds a given threshold. Also, attraction and rejection potential field forces are calculated using the equations proposed by [7].

- **Package:** `potential_fields`
- **Parameters:** `debug`, `show_image`, `use_pot_fields`, `use_lidar`, `use_point_cloud`, `laser_max/min_x/y/z`, `cloud_max/min_x/y/z`, `laser_pot_fields_d0`, `laser_pot_fields_k_rej`, `cloud_pot_fields_d0`, `cloud_pot_fields_k_rej`, `cloud_downsampling`, `cloud_points_threshold`, `lidar_points_threshold`, `point_cloud_topic`, `laser_scan_topic`

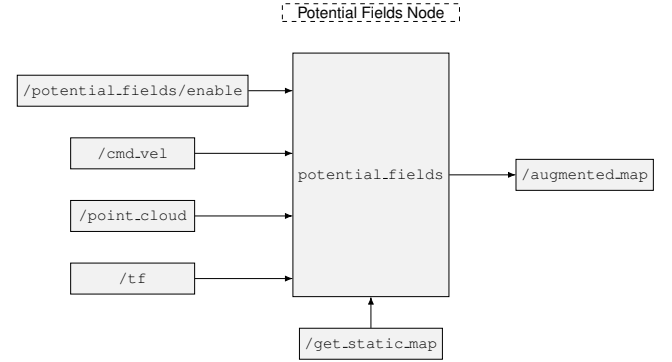


Fig. 4: The dynamic obstacle detector module in the Map Augmenter package. Block diagram of the dynamic obstacle detector module in the `potential_fields` package, which enhances the environment representation by integrating real-time sensor data to detect and insert dynamic obstacles into the map.

V. EXPERIMENTAL SETUP AND RESULTS

The proposed system was evaluated using the Toyota Human Support Robot (HSR) [8]. The experimental environment was a domestic setting furnished with standard household furniture [3]. A 2D occupancy map of the environment was generated beforehand using a 2D LiDAR sensor mounted at a fixed height on the robot.

To assess the system's performance under real-world conditions, a set of small objects (with heights below the LiDAR plane) were distributed across the environment. Additionally, various pieces of furniture, such as tables and chairs, were used to simulate occluded obstacles. In these cases, the LiDAR sensor is only able to detect the base or legs of the objects, while the upper surfaces remain invisible to the navigation system based solely on LiDAR data. This limitation is illustrated in Figure 5.

We conducted baseline tests using the standard ROS 2 navigation stack, Nav2, for point-to-point navigation. As expected, the LiDAR-based system failed to detect many of the occluded or partially visible obstacles, resulting in several collisions.



Fig. 5: HSR robot operating in a domestic environment with obstacles of varying heights. Note the presence of small objects on the floor and partially occluded furniture surfaces.

In contrast, our approach employs a dual-map strategy:

- A **robot map**, used exclusively for localization, relying on the same algorithm as Nav2—namely, `amcl`.
- A **prohibition map**, which is used for path planning and can be manually annotated with regions or obstacles that are invisible to the robot’s sensors. This separation ensures that modifications for planning do not degrade localization performance.

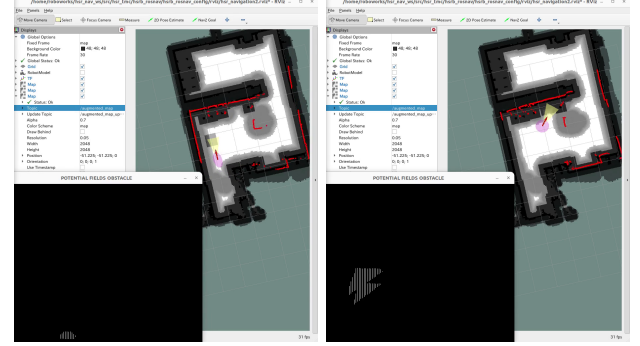
Furthermore, the system integrates an RGB-D camera mounted on the robot’s head to detect obstacles at varying heights. When an obstacle is detected outside the LiDAR field of view, the collision risk detector module is triggered, and temporary obstacle data is added to the costmap through the `map augementer` module. This enables the path planner to replan trajectories that avoid such obstacles. Figure 6 illustrates this behavior: the robot identifies both a low object (Figure 6a) and a higher obstacle (Figure 6b), not visible to the LiDAR.

To further enhance robustness, the system employs a potential fields module, which combines LiDAR and RGB-D point cloud data to perform real-time reactive navigation. This module applies repulsive forces when sudden nearby obstacles—such as sharp corners or previously occluded dynamic objects—are detected, allowing the robot to safely avoid collisions and continue toward its goal.

VI. CONCLUSION AND FUTURE WORK

This work presented a modular and extensible framework for dynamic map enhancement and real-time navigation in robotic systems. At the core of this architecture is the `map augementer` node, responsible for fusing static maps with real-time sensory inputs such as point clouds and TF data to generate augmented cost maps. These enriched representations allow for dynamic response to environmental changes and form the basis for path planning and obstacle avoidance.

A key feature of our system is the use of potential fields to enable behavior-based avoidance of unexpected obstacles. The robot navigates the environment by following the negative gradient of the potential function, effectively descending



(a)

(b)

Fig. 6: Obstacle detection using the RGB-D camera. The left and right panels show a window displaying the output of the 3D obstacle detector and RViz2 with temporarily added obstacles from the map augementer. (a) Detection of a small ball below the LiDAR height. (b) Detection of a tabletop surface above the LiDAR height

toward the goal while being repelled from obstacles. This method allows for smooth, reactive movement and is particularly well-suited for integrating dynamic sensor updates like those provided by our obstacle detection pipeline.

The modular ROS-based architecture ensures flexibility, allowing the framework to be tailored to different robot platforms and sensor configurations. Overall, this system provides a robust foundation for autonomous navigation in unstructured and evolving environments. Future work will focus on extending the potential field behavior to multi-agent scenarios and benchmarking the system under different dynamic conditions and map complexities.

REFERENCES

- [1] L. Contreras and H. Okada, “Personal and Domestic Robotics,” *Encyclopedia of Robotics*, 2022.
- [2] “RoboCup at Home,” <https://athome.robocup.org/>. Accessed: 2025-07-01.
- [3] L. Contreras, T. Yamamoto, Y. Matsusaka, and H. Okada, “Towards general purpose service robots: World Robot Summit Partner Robot Challenge,” *Advanced Robotics*, 2022.
- [4] F. Amigoni, E. Bastianelli, J. Berghofer, A. Bonarini, G. Fontana, N. Hochgeschwender, L. Iocchi, G. Kraetzschmar, P. Lima, M. Matteucci, P. Miraldo, D. Nardi, and V. Schiaffonati, “Competitions for benchmarking: Task and functionality scoring complete performance assessment,” 2015.
- [5] M. Negrete, J. Savage, and L. Contreras, “A Motion-Planning System for a Domestic Service Robot,” *SPIIRAS Proceedings*, vol. 60, no. 5, pp. 5–38, 2018.
- [6] “Pumas Navigation,” https://github.com/ARTenshi/robot_navigation. Accessed: 2025-07-01.
- [7] J. A. Herrera Ortiz, K. Rodríguez-Vázquez, M. A. Padilla Castaneda, and F. Arámbula Cosío, “Autonomous robot navigation based on the evolutionary multi-objective optimization of potential fields,” *Engineering Optimization*, vol. 45, no. 1, pp. 19–43, 2013.
- [8] T. Yamamoto, K. Terada, A. Ochiai, F. Saito, Y. Asahara, and K. Murase, “Development of Human Support Robot as the research platform of a domestic mobile manipulator,” *ROBOMECH Journal*, vol. 6, 2019.